UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Fundada en 1551

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE – LIMA



Implementación y administración de un servidor web con Ubuntu Server y Nginx

Actividad práctica: servidor backend y persistencia

Grupo 04

Estudiantes:

Joshua Nicolas Chavez Cerna

Guevara Chávez Luis Rodrigo (coordinador)

Montes Ramos Carol

Toro Vargas Karlo Andrés

Núñez Cardenas Ivan Joaquin

Docente:

Candela Arce, David Felix

Asignatura:

Taller de construcción de software web

Lima — 2026

Índice

Implementación y administración de un servidor web con Ubuntu Server y Nginx 3
1. Introducción 3
2. Objetivos 3
3. Requisitos de hardware y software 3
4. Arquitectura implementada 5
5. Procedimiento por fases 5
6. Comandos utilizados y su función 9
7. Capturas de pantalla 10
8. Resultados de las pruebas 16
9. Problemas encontrados y soluciones 17
10. Respuestas a las preguntas de análisis 17
11. Conclusiones 19
12. Recomendaciones 19
Anexo A — Estructura del proyecto 20

Implementación y administración de un servidor web con Ubuntu Server y Nginx

Actividad práctica — Servidor backend y persistencia Institución: Universidad Nacional Tecnológica del Pacífico (sitio de demostración) **Servidor:** `srvuniversidad` — 192.168.150.10 **Fecha:** 31 de octubre de 2026

1. Introducción

La presente práctica documenta la implementación completa de un servidor web sobre Ubuntu Server 24.04.5 LTS ejecutándose en una máquina virtual sobre Microsoft Hyper-V. El trabajo abarca el ciclo de vida completo de un servidor: aprovisionamiento del hardware virtual, instalación del sistema operativo, configuración de red, publicación de un sitio web con Nginx, administración de usuarios y permisos, supervisión de recursos, revisión de registros, copias de seguridad y prueba de restauración.

El escenario elegido es el sitio web de una universidad ficticia, lo que permite trabajar con contenido realista (presentación institucional, misión, visión, oferta académica y datos de contacto) sin depender de servicios de terceros.

Todas las verificaciones incluidas en este informe fueron ejecutadas sobre el sistema real; no se presentan resultados simulados.

2. Objetivos

Objetivo general

Implementar, configurar y administrar un servidor web funcional sobre Ubuntu Server con Nginx, aplicando procedimientos de supervisión, seguridad y mantenimiento.

Objetivos específicos

1. Aprovisionar una máquina virtual con recursos definidos (2 vCPU, 4 GB de RAM, 20 GB de disco).
2. Instalar Ubuntu Server LTS y configurar identidad de red (hostname e IP estática).
3. Instalar y configurar Nginx como servidor web y publicar un sitio institucional.
4. Crear un usuario administrativo adicional con permisos mínimos sobre el contenido web.
5. Supervisar el consumo de CPU, memoria y disco, y verificar el estado del servicio.
6. Analizar los registros de acceso y de errores de Nginx.
7. Generar copias de seguridad y validar la restauración del sitio.
8. Elaborar la documentación técnica con evidencias de cada etapa.

3. Requisitos de hardware y software

3.1 Anfitrión

Componente	Característica
Equipo	MSI MS-7E64
Procesador	Intel Core Ultra 5 225F

Componente	Característica
Memoria RAM	16 GB (15,63 GB utilizables)
Almacenamiento libre	341 GB en la unidad C:
Sistema operativo	Windows 11 Pro 26H2 (build 10.0.26300)
Hipervisor	Microsoft Hyper-V (servicios `vmms`, `vmcompute`, `HvHost` activos)

3.2 Máquina virtual

Recurso	Configuración
Nombre	`srv-universidad` (nombre del sistema invitado: `srvuniversidad`)
Generación	2 (UEFI)
vCPU	2
Memoria	4096 MB estáticos
Disco	VHDX de 20 GB, crecimiento dinámico
Firmware	Secure Boot desactivado
Red	Conmutador interno `srv-lab` con NAT
Ubicación	`C:\Hyper-V\srv-universidad\`

3.3 Software

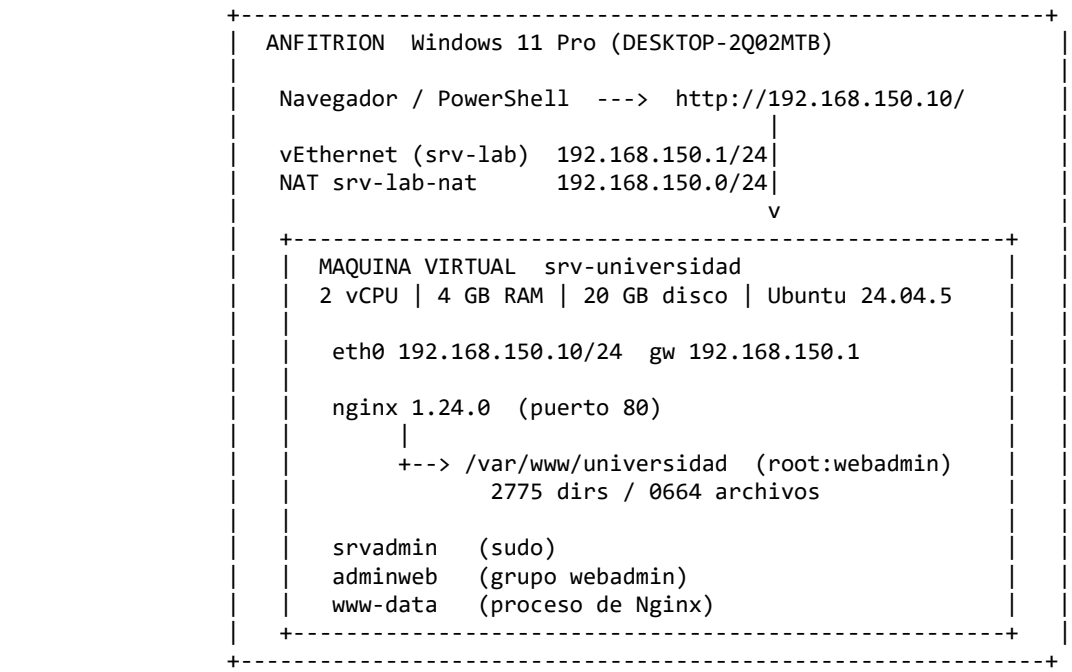
Software	Versión
Ubuntu Server LTS	24.04.5
Kernel	6.8.0-139-generic
Nginx	1.24.0-2ubuntu7.19
OpenSSH Server	paquete de la distribución
Netplan	configuración declarativa de red

3.4 Imagen utilizada

```
ubuntu-24.04.5-live-server-amd64.iso   (4 080 486 400 bytes)
SHA-256: 97f3d7ffb032c3eb3b23d2c8be9cc76e60c2c1f2c0146ba5ba9fe01cafae0fd8
```

La huella calculada con `Get-FileHash` coincide con la publicada en `releases.ubuntu.com`, lo que garantiza la integridad de la imagen antes de instalar.

4. Arquitectura implementada



Decisiones de diseño

- **Conmutador interno con NAT:** aísla el servidor de la red doméstica real (192.168.100.0/24) y evita conflictos de direcciones, a la vez que le proporciona salida a Internet para actualizaciones.
- **IP estática:** requerida para ofrecer servicios de forma predecible y para documentar un punto de acceso estable.
- **Sitio virtual propio:** en lugar de reutilizar `/var/www/html`, se crea `/etc/nginx/sites-available/universidad` con registros de log independientes, lo que facilita el diagnóstico y la auditoría.

5. Procedimiento por fases

Fase 1 — Máquina virtual e instalación

1. Verificación del entorno: hipervisor disponible, espacio en disco, integridad de la ISO.
2. Creación del conmutador interno `srv-lab` y del NAT `srv-lab-nat`.
3. Creación de la VM en Hyper-V con los recursos definidos.
4. Instalación de Ubuntu Server 24.04.5 LTS con:
    - teclado Spanish (Latin American),
    - disco completo con LVM,
    - hostname del sistema,
    - usuario administrador `srvadmin`,
    - **OpenSSH Server** habilitado.

Fase 2 — Red y hostname

Parámetro	Valor
Hostname	`srvuniversidad`
Interfaz	`eth0`
Dirección IPv4	`192.168.150.10/24`
Puerta de enlace	`192.168.150.1`
DNS	`8.8.8.8`, `1.1.1.1`

Configuración declarativa en `/etc/netplan/01-netcfg.yaml`:

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: false
      addresses: [192.168.150.10/24]
      routes:
        - to: default
          via: 192.168.150.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Se desactiva la gestión de red por `cloud-init` mediante `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` para que la configuración no se sobrescriba en los siguientes arranques.

Fase 3 — Nginx y publicación del sitio

1. `apt-get update` e instalación del paquete `nginx`.
2. Publicación del contenido en `/var/www/universidad/` (`index.html`, `estilos.css`, `favicon.svg`), propietario `root:www-data` y permisos 644.
3. Creación del sitio virtual `/etc/nginx/sites-available/universidad`:

```
server {
    listen 80;
    listen [::]:80;
    server_name srvuniversidad universidad.local www.universidad.local;

    root /var/www/universidad;
    index index.html;
    access_log /var/log/nginx/universidad.access.log;
    error_log  /var/log/nginx/universidad.error.log;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~* \.(css|js|svg|png|jpe?g|ico|woff2?)$ {
        expires 7d;
        access_log off;
    }
}
```

4. Habilitación del sitio, retirada del sitio por defecto, validación con `nginx -t` y recarga del servicio.

Contenido del sitio: presentación institucional, misión, visión, oferta académica de seis programas de pregrado, datos de contacto ficticios y nota de pie aclarando el carácter académico del contenido.

Fase 4 — Usuarios y permisos

Elemento	Configuración
Grupo	`webadmin`
Usuario	`adminweb` (home propio, shell `/bin/bash`, miembro de `webadmin`)
Propietario del sitio	`root:webadmin`
Directorios	`2775` (rwxrwsr-x — bit setgid activo)
Archivos	`0664` (rw-rw-r–)
`umask` de adminweb	`002`

Criterio de seguridad: no se utiliza `777` en ningún caso. El bit `setgid` en los directorios hace que todo archivo nuevo herede el grupo `webadmin`; el permiso de lectura para «otros» permite que `www-data` acceda al contenido sin pertenecer al grupo de edición.

Pruebas realizadas:

- `adminweb` crea un archivo dentro del sitio → correcto, propietario `adminweb:webadmin`.
- Nginx sirve ese archivo con `HTTP 200` → correcto.
- `adminweb` intenta escribir en `/etc` → **denegado**.

Fase 5 — Supervisión

Recurso	Valor observado
Carga media	0.00, 0.00, 0.00
vCPU	2 (Intel Core Ultra 5 225F)
Procesos	112 (1 en ejecución)
Memoria total	3915.3 MiB
Memoria usada	420.7 MiB (≈11 %)
Memoria disponible	3494.6 MiB
Swap	sin configurar (0 B)
Raíz `/`	17 GB, 2.7 GB usados (17 %), 14 GB libres
`/boot`	1.7 GB, 105 MB usados (7 %)
Contenido web	32 KB en `/var/www/universidad`
Servicio nginx	`active (running)`, `enabled`
Procesos nginx	1 maestro + 2 trabajadores
Memoria del servicio	2.6 MB (pico 5.3 MB)
Puerto 80	escuchando en IPv4 e IPv6

Tráfico generado desde el anfitrión y registrado en el log de acceso:

```
192.168.150.1 - - "GET / HTTP/1.1" 200 9085
192.168.150.1 - - "GET /estilos.css HTTP/1.1" 200
192.168.150.1 - - "GET /pagina-inexistente-1 HTTP/1.1" 404 162
```

Fase 6 — Copia de seguridad y restauración

Etapa	Resultado
Directorio de copias	`/var/backups/universidad` (permisos 750)
Archivo generado	`universidad-20261031-182121.tar.gz` (5.5 KB)
Huella SHA-256	`fb1e734b00aa811441ff7b35383b967316ed48c1219ea3e040a8edc56adbb5c3`
Verificación de integridad	`OK` (`sha256sum -c`)
Contenido verificado	`universidad/index.html`, `estilos.css`, `favicon.svg`
Simulación de pérdida	`rm index.html` → `HTTP 403`
Restauración	extracción del tar y reinstalación con propietario y permisos
Verificación posterior	`HTTP 200`, 9085 bytes
Comparación de huellas	idénticas (`efc9b6d5…5e705ed2`)

Fase 7 — Pruebas finales

N.º	Verificación	Resultado
1	Ubuntu Server instalado y operativo	Cumplido
2	Hostname configurado (`srvuniversidad`)	Cumplido
3	Red funcional (IP estática, gateway, DNS, salida a Internet)	Cumplido
4	Nginx activo y habilitado al arranque	Cumplido
5	Página accesible desde el navegador del anfitrión	Cumplido (HTTP 200)
6	Usuario adicional `adminweb` creado	Cumplido
7	Permisos sin 777 y con escritura de grupo controlada	Cumplido
8	CPU, memoria y disco supervisados	Cumplido
9	Logs de acceso y error revisados	Cumplido
10	Backup creado y restauración comprobada	Cumplido

6. Comandos utilizados y su función

Comando	Función
`Get-VMSwitch`, `New-VMSwitch -SwitchType Internal`	Inventario y creación del conmutador virtual
`New-NetNat -InternalIPInterfaceAddressPrefix 192.168.150.0/24`	Salida a Internet para la subred del laboratorio
`New-VM -Generation 2 -MemoryStartupBytes 4GB -NewVHDSizeBytes 20GB`	Creación de la VM
`Set-VMProcessor -Count 2`	Asignación de vCPU
`Set-VMFirmware -BootOrder`	Orden de arranque (DVD antes que disco)
`hostnamectl`	Nombre del sistema
`ip -4 addr show`, `ip route`	Verificación de direcciones y rutas
`netplan apply`	Aplicación de la configuración de red
`apt-get update`, `apt-get install nginx`	Actualización e instalación del servidor web
`nginx -t`	Validación de la sintaxis antes de aplicar cambios
`systemctl enable/reload nginx`	Persistencia y recarga sin cortar el servicio
`groupadd`, `useradd`, `usermod -aG`	Gestión de grupos y usuarios
`chown -R root:webadmin`, `chmod 2775`, `chmod 0664`	Permisos mínimos necesarios
`top -bn1`, `free -h`, `df -hT`, `du -sh`	Supervisión de CPU, memoria y disco
`systemctl status nginx`	Estado del servicio
`journalctl -u nginx`	Eventos del servicio
`tail -n 10 …/universidad.access.log`	Registros de acceso
`tar -czf`, `sha256sum`, `sha256sum -c`	Copia comprimida y verificación de integridad
`curl -o /dev/null -w "%{http_code}"`	Pruebas HTTP objetivas
`ss -ltnp '( sport = :80 )'`	Puertos en escucha

7. Capturas de pantalla

7.1 Portada del sitio publicado

UNTP
Universidad Nacional Tecnológica del Pacífico
Institución
Misión y visión
Oferta académica
Contacto
ADMISIÓN 2027 · MODALIDADES REGULAR Y TRASLADO
Ingeniería y ciencia
al servicio del país
Formamos profesionales capaces de resolver problemas reales con tecnología, rigor científico y responsabilidad social. Más de 40 años de trayectoria académica.
Ver carreras
Solicitar información
ESTUDIANTES
8 400
DOCENTES
512
LABORATORIOS
36
CONVENIOS
92
PRESENTACIÓN INSTITUCIONAL
Una universidad pública orientada a la tecnología
La Universidad Nacional Tecnológica del Pacífico (UNTP) es una institución pública de educación superior dedicada a la formación de ingenieros, científicos y gestores. Su modelo académico combina fundamentos teóricos sólidos con práctica intensiva en laboratorio desde el primer ciclo.
La universidad articula docencia, investigación aplicada y extensión social. Sus líneas de investigación prioritarias son transformación digital, energías renovables, gestión del agua y salud pública.
Licenciamiento institucional vigente
Laboratorios equipados por especialidad
Programa de movilidad estudiantil nacional e internacional
Bolsa de prácticas con empresas del sector productivo
Datos institucionales
Fundación 1983
Facultades 4
Programas 11
Sede Campus Universitario, Lima
Modalidad Presencial · Semipresencial
IDENTIDAD
Misión y visión
Misión
Formar profesionales competentes, éticos y críticos en las áreas de ingeniería, ciencia y gestión, mediante una educación de calidad, investigación aplicada y vinculación permanente con la sociedad.
Visión
Ser una universidad de referencia nacional e internacional por la excelencia de su formación tecnológica, el impacto de su investigación y su contribución al desarrollo sostenible del país.
OFERTA ACADÉMICA
Seis de nuestros once programas
Todos los programas tienen una duración de diez semestres académicos y otorgan el grado de bachiller y el título profesional correspondiente.
INGENIERÍA
Ingeniería de Sistemas
Arquitectura de software, bases de datos, gestión de servicios TI.
10 semestres · Presencial
INGENIERÍA
Ingeniería de Software
Desarrollo ágil, calidad de software, aplicaciones distribuidas.
10 semestres · Presencial
INGENIERÍA
Ciencia de Datos
Estadística aplicada, aprendizaje automático, visualización de datos.
10 semestres · Presencial
INGENIERÍA
Redes y Ciberseguridad
Infraestructura de redes, seguridad informática, centros de datos.
10 semestres · Presencial
INGENIERÍA
Ingeniería Industrial
GESTIÓN
Administración de Empresas

Figura 1. Página principal servida por Nginx, capturada en el navegador del anfitrión mediante http://192.168.150.10/

La captura corresponde a la renderización real del sitio desde el anfitrión. Se observa la cabecera con el nombre de la universidad, el bloque principal de presentación, la sección de datos institucionales, misión y visión, la oferta académica y los datos de contacto.

7.2 Verificación del sitio tras la restauración



Figura 2. Sitio operativo después de restaurar index.html desde la copia de seguridad

Esta captura se tomó después de eliminar `index.html` y restaurarlo desde el archivo `universidad-20261031-182121.tar.gz`. Confirma que el procedimiento de recuperación devuelve el servicio a su estado funcional.

7.3 Evidencias de consola

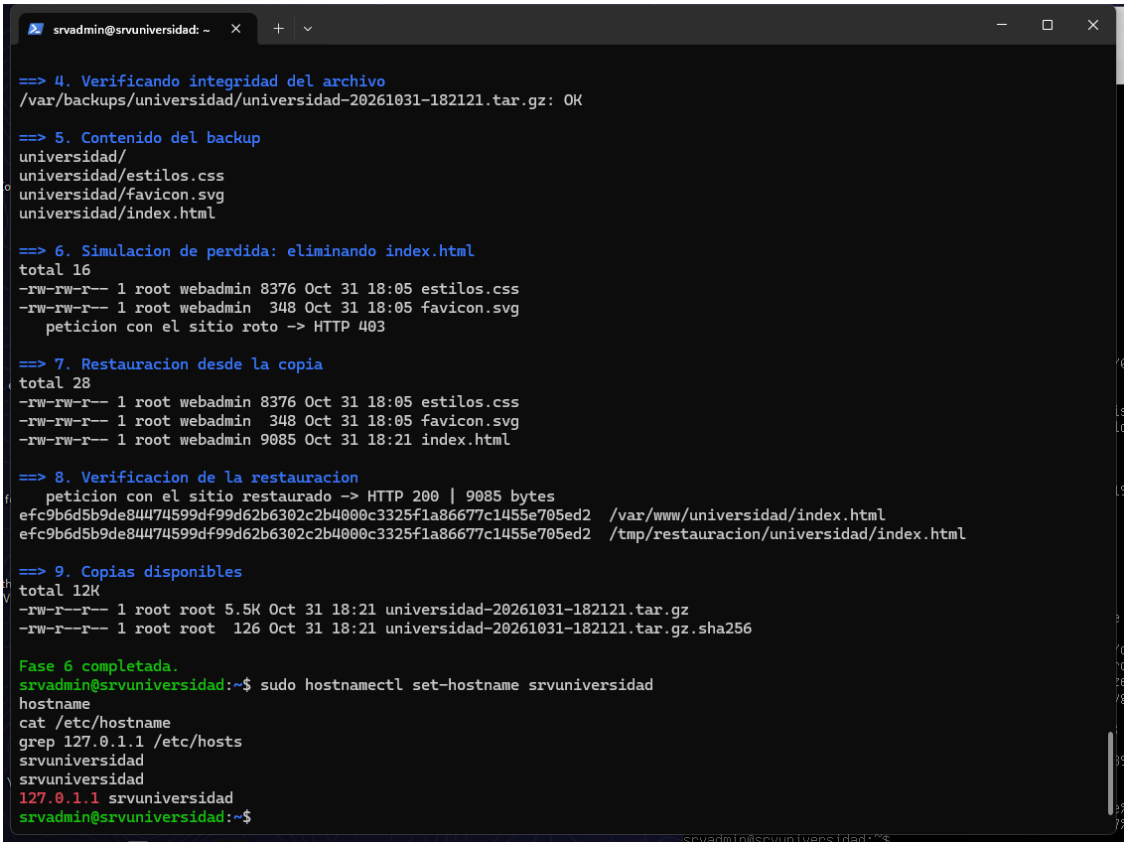


Figura 3. Fase 6: verificación del backup, caída simulada (HTTP 403), restauración (HTTP 200) y comprobación del hostname

La sesión SSH real muestra la secuencia completa de la prueba de recuperación: `sha256sum -c` devuelve `OK`, el sitio responde `403` al eliminar `index.html`, y tras extraer la copia vuelve a `200` con 9085 bytes. Las dos huellas SHA-256 del archivo restaurado son idénticas. En la misma sesión se confirma que `hostname`, `/etc/hostname` y la entrada `127.0.1.1` de `/etc/hosts` coinciden en `srvuniversidad`.

7.4 Administración de usuarios y permisos

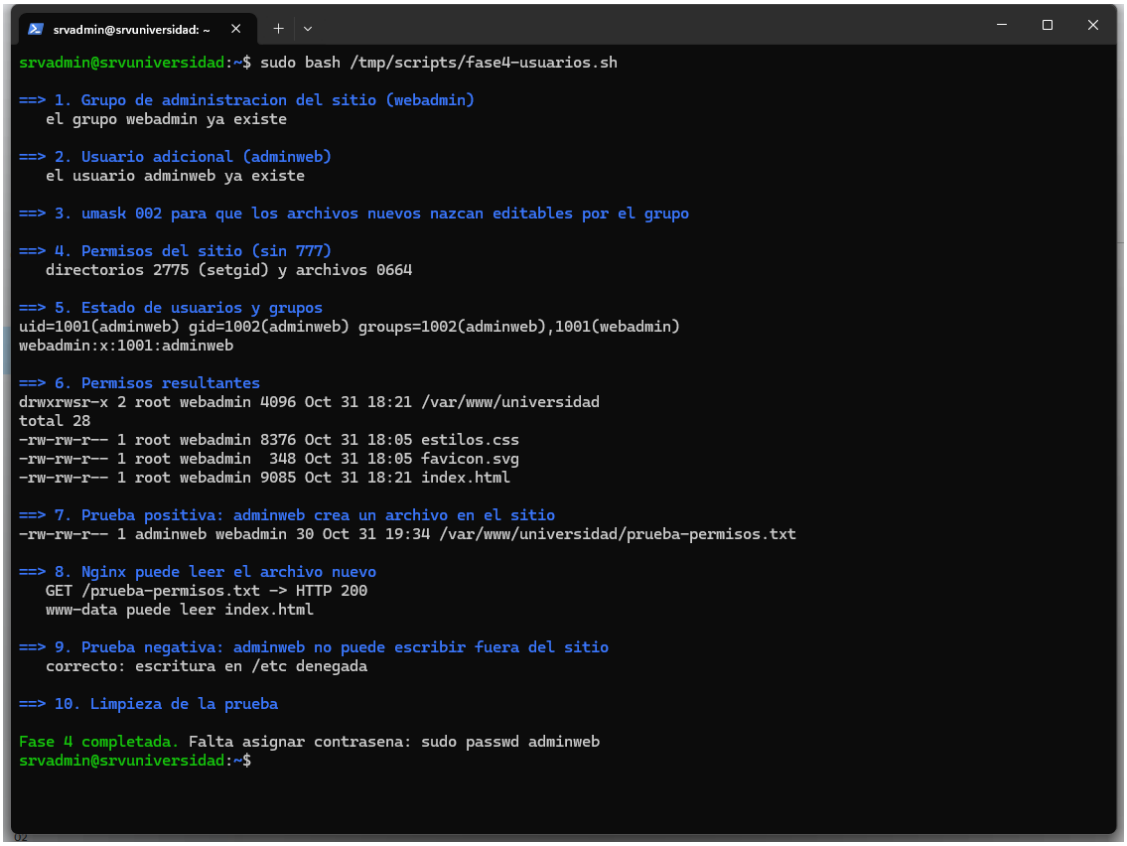


Figura 4. Fase 4: grupo webadmin, usuario adminweb, permisos 2775/0664 y pruebas de acceso

La captura evidencia el resultado de la política de permisos aplicada sobre `/var/www/universidad`:

- `id adminweb` → `uid=1001(adminweb)`, miembro del grupo `webadmin`.
- `ls -ld` → `drwxrwsr-x`, es decir, **2775** con el bit setgid activo (`s` en el grupo).
- Los archivos publicados aparecen como `-rw-rw-r--` (**0664**) con propietario `root:webadmin`.
- **Prueba positiva:** `adminweb` crea `prueba-permisos.txt` y Nginx lo sirve con `HTTP 200`.
- **Prueba negativa:** `adminweb` intenta escribir en `/etc` y el sistema lo **deniega**.

Ningún recurso del sitio utiliza permisos `777`.

7.5 Supervisión de recursos y estado del servicio

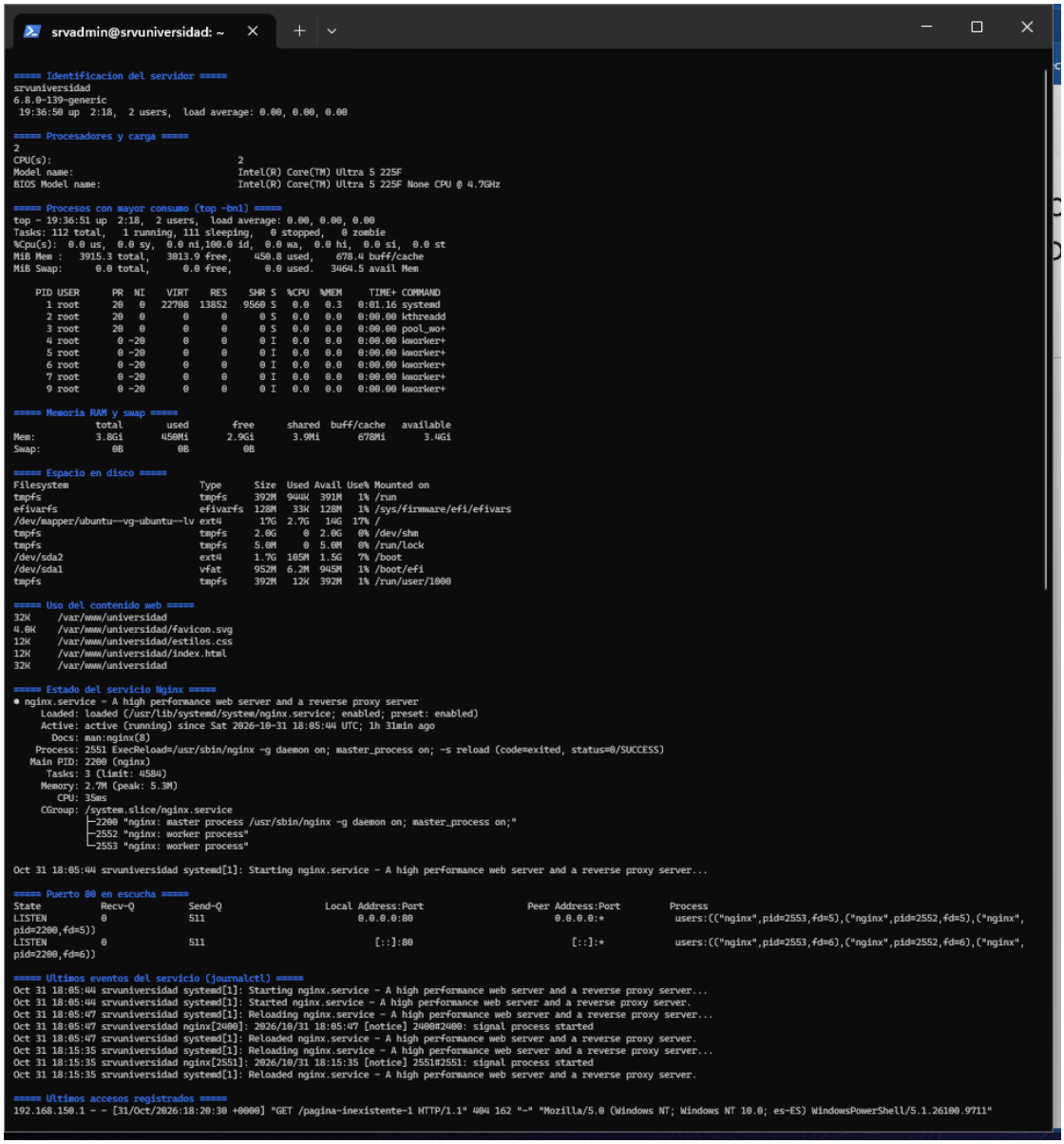


Figura 5. Fase 5: identificación del servidor, CPU, procesos, memoria, disco, estado de Nginx, puerto 80 y eventos del servicio

La captura reúne los indicadores solicitados en una sola sesión:

Indicador	Valor visible
Hostname	`srvuniversidad`
Kernel	`6.8.0-139-generic`
Tiempo encendido	2 h 18 min
Carga media	0.00, 0.00, 0.00
vCPU	2 (Intel Core Ultra 5 225F)
Procesos	112, ninguno en estado *zombie*

Indicador	Valor visible
Memoria	3915.3 MiB totales, 450.8 MiB usados, 3404.5 MiB disponibles
Swap	0 B (sin configurar)
Raíz `/`	17 G, 2.7 G usados (17 %), 14 G libres
Contenido web	32 K en `/var/www/universidad`
Servicio Nginx	`active (running)`, `enabled`, 1 maestro + 2 workers
Puerto 80	escucha en IPv4 e IPv6
Journal	recargas de configuración sin errores

El detalle de los registros de acceso se documenta en la sección 8, donde se incluyen las 22 peticiones generadas desde el anfitrión con códigos `200` y `404`.

> Nota: la captura de la instalación de Nginx y de la validación con `nginx -t` no se conservó como imagen; no obstante, su resultado queda registrado en la sección 8 (tabla de resultados) y en la configuración documentada en la sección 5.

8. Resultados de las pruebas

Prueba	Resultado esperado	Resultado obtenido
Resolución de la ISO	SHA-256 igual al oficial	Coincide
Conectividad anfitrión → servidor	0 % de pérdida	4/4 paquetes, TTL 64
Puerto 22	Abierto	`TcpTestSucceeded = True`
Puerto 80	Abierto	`LISTEN 0.0.0.0:80`
Petición HTTP al sitio	200 + contenido propio	200, 9085 bytes, título correcto
Rutas inexistentes	404	404 (registradas en el log de acceso)
Persistencia del servicio	`enabled`	`enabled`
Escritura de adminweb en el sitio	Permitida	Permitida
Escritura de adminweb en `/etc`	Denegada	Denegada
Integridad del backup	`OK`	`OK`
Sitio tras borrar `index.html`	Error	403
Sitio restaurado	200	200, 9085 bytes
Huellas del archivo restaurado	Idénticas	Idénticas

9. Problemas encontrados y soluciones

Problema	Causa	Solución aplicada
La VM no arrancaba por falta de memoria	El anfitrión tenía 3.07 GB libres y la VM solicitaba 4 GB	Se liberaron aplicaciones del anfitrión antes de iniciar el servidor
La VM arrancaba por red en lugar del instalador	El orden por defecto priorizaba la tarjeta de red	`Set-VMFirmware -BootOrder` con el DVD en primer lugar
MAC estática `000000000000`	Dirección fija asignada por defecto	`Set-VMNetworkAdapter -DynamicMacAddress`
Interfaz `eth0` sin dirección IP	No existía archivo de netplan tras la instalación	Configuración declarativa en `/etc/netplan/01-netcfg.yaml` y bloqueo de `cloud-init`
Solo 10 GB de raíz en un disco de 20 GB	El instalador creó un LV menor que el grupo de volúmenes	`lvextend -l +100%FREE` y `resize2fs` en caliente
El log de errores de Nginx permanecía vacío	Los 404 provienen de `try_files … =404`, que no intenta abrir un archivo inexistente	Se documenta que ese evento solo aparece en el log de acceso
El sitio quedó inaccesible tras borrar `index.html`	Comportamiento esperado con `autoindex off`	Restauración desde la copia de seguridad

10. Respuestas a las preguntas de análisis

10.1 ¿Por qué es importante supervisar permanentemente los servidores?

Porque un servidor es un sistema en cambio constante y los fallos rara vez son instantáneos: se manifiestan como tendencias. La supervisión permite detectar **degradación gradual** (memoria que crece sin liberarse, disco que se llena por logs, CPU que sube por un proceso descontrolado) antes de que el servicio se caiga.

En esta práctica se comprobó con datos concretos: 420.7 MiB de memoria usada, carga 0.00 y 17 % de disco ocupado constituyen una **línea base**. Cualquier desviación significativa respecto de esa base es una señal de alerta temprana. Sin línea base, no hay forma de distinguir lo normal de lo anómalo.

Además, la supervisión sostiene tres compromisos: **disponibilidad** (el servicio responde), **rendimiento** (responde a tiempo) y **capacidad** (seguirá respondiendo cuando crezca la demanda). Un servidor sin supervisión se administra por reacción, siempre después de que el usuario reporte el problema.

10.2 ¿Qué consecuencias puede ocasionar una mala administración de permisos?

Las consecuencias se agrupan en tres dimensiones:

- **Seguridad:** permisos excesivos como `777` permiten que cualquier usuario —o un proceso comprometido— modifique el contenido publicado. Un atacante que altere `index.html` puede ejecutar *defacement* o inyectar código malicioso que afecte a todos los visitantes. Peor aún, si el directorio web es escribible por el usuario con el que corre el servidor, se habilita la ejecución de scripts arbitrarios.
- **Integridad:** sin control de propiedad, dos administradores pueden sobrescribirse mutuamente y resulta imposible reconstruir quién cambió qué.

- **Operación:** permisos demasiado restrictivos rompen el servicio (por ejemplo, `www-data` sin lectura devuelve `403`), mientras que permisos demasiado amplios se vuelven invisibles hasta que ocurre el incidente.

La solución aplicada en la práctica ilustra el equilibrio: `root:webadmin` con `2775` y `0664`. `adminweb` escribe porque pertenece al grupo; `www-data` lee porque el permiso de «otros» incluye lectura; nadie fuera del grupo puede modificar el sitio. El principio es **mínimo privilegio**: cada identidad recibe exactamente lo que necesita.

10.3 ¿Qué sucede cuando un servidor web deja de funcionar?

De forma inmediata, el sitio deja de responder y los usuarios reciben errores de conexión o páginas de error. Pero el impacto real es en cascada:

- **Técnico:** se pierden peticiones en curso, se interrumpen sesiones y se degradan los sistemas que dependen del sitio (formularios, APIs, correo institucional).
- **Económico:** cada minuto de inactividad puede traducirse en pérdida de matrículas, ventas o credibilidad.
- **Reputacional:** los usuarios asocian la caída con falta de profesionalismo, y la recuperación de la confianza es más lenta que la reparación técnica.
- **Operativo:** se activa un ciclo de urgencia donde se diagnostica sin datos, porque si no había supervisión ni logs revisados, no hay evidencia de qué ocurrió.

En esta práctica se reprodujo una caída controlada eliminando `index.html`: Nginx respondió `403` de inmediato y el servicio siguió activo. Esa distinción es clave: **el proceso puede estar «activo» y el sitio estar «caído»**, por eso la supervisión debe incluir pruebas funcionales (una petición HTTP real), no solo el estado del servicio.

10.4 ¿Por qué es necesario realizar copias de seguridad?

Porque ninguna medida de prevención es infalible: los discos fallan, los archivos se borran por error, un despliegue defectuoso sobrescribe contenido válido y un ataque de *ransomware* cifra los datos. La copia de seguridad es la única garantía de **recuperación** cuando todo lo demás falla.

En la práctica se comprobó su valor de forma tangible: se eliminó `index.html`, el sitio pasó a `403` y se restauró en segundos con `HTTP 200` y huella SHA-256 idéntica.

Tres lecciones derivadas del ejercicio:

1. **Una copia no verificada no es una copia.** Por eso se calcula y comprueba el SHA-256 y se lista el contenido con `tar -tzf`.
2. **Una copia que nunca se restauró es una hipótesis.** La única prueba válida es ejecutar la restauración completa.
3. **La copia debe ser identificable:** el nombre incluye fecha y hora (`universidad-20261031-182121.tar.gz`) para poder elegir el punto de recuperación.

10.5 ¿Qué ventajas ofrece administrar servidores de manera remota?

- **Elimina la dependencia física:** el servidor puede estar en un centro de datos y administrarse desde cualquier ubicación con conectividad.
- **Reduce el tiempo de respuesta:** un problema se diagnostica en segundos por SSH en lugar de requerir desplazamiento.

- **Permite automatización:** los comandos remotos pueden scriptearse y programarse, lo que hace las operaciones repetibles y auditables.
- **Habilita el trabajo en equipo:** varios administradores acceden con identidades propias, lo que mejora la trazabilidad frente a una única cuenta local.
- **Disminuye riesgos operativos:** no se interviene físicamente el hardware ni se expone la consola a personas no autorizadas.

En esta práctica el acceso remoto por SSH fue determinante: la consola de la máquina virtual solo permite copiar y pegar con dificultad, mientras que una sesión SSH posibilita pegar comandos largos, transferir archivos con `scp` y revisar registros con comodidad. Como contrapartida, la administración remota exige controles adicionales: autenticación robusta (claves en lugar de contraseñas), restricción por firewall y registro de accesos.

11. Conclusiones

1. Se implementó un servidor web funcional sobre Ubuntu Server 24.04.5 LTS en Hyper-V, con recursos definidos y verificados: 2 vCPU, 4 GB de RAM y 20 GB de disco.
2. La red quedó configurada de forma declarativa y persistente mediante Netplan, con dirección estática, puerta de enlace y DNS, lo que garantiza un punto de acceso estable para el servicio.
3. Nginx 1.24.0 sirve el sitio institucional desde `/var/www/universidad` con un sitio virtual propio, cabeceras de seguridad y registros independientes. La validación previa con `nginx -t` permitió aplicar cambios sin interrumpir el servicio.
4. La administración de permisos se resolvió con el principio de mínimo privilegio: `root:webadmin`, directorios `2775` con setgid y archivos `0664`. Se verificó tanto la capacidad de edición del usuario `adminweb` como la denegación de escritura fuera del sitio.
5. La supervisión permitió establecer una línea base de consumo (≈11 % de memoria, 17 % de disco, carga 0.00) y confirmar el estado del servicio y del puerto 80.
6. El análisis de logs demostró su utilidad para auditar tráfico real: 22 peticiones desde el anfitrión, con códigos 200 y 404 claramente diferenciados.
7. La copia de seguridad comprimida, con huella SHA-256 y restauración probada, demostró ser un mecanismo eficaz: el sitio pasó de `403` a `200` con contenido idéntico.
8. El proyecto confirma que la administración de servidores es un ciclo continuo: instalar es solo el inicio; supervisar, registrar y respaldar es lo que sostiene el servicio en el tiempo.

12. Recomendaciones

1. **Automatizar las copias de seguridad** con `cron` o `systemd timers`, aplicando una política de retención (por ejemplo, conservar 7 copias diarias y 4 semanales).
2. **Sustituir la autenticación por contraseña en SSH por claves públicas**, y deshabilitar `PasswordAuthentication` una vez configuradas.
3. **Habilitar un firewall** (`ufw`) permitiendo únicamente los puertos 22 y 80.
4. **Habilitar HTTPS** con certificados (Let's Encrypt o una CA interna), redirigiendo el tráfico HTTP y añadiendo HSTS.
5. **Configurar la rotación de logs** (`logrotate`) para evitar el crecimiento indefinido de los archivos de registro.
6. **Incorporar una herramienta de supervisión continua** con alertas (Netdata, Zabbix, Prometheus + Node Exporter) en lugar de consultas manuales.
7. **Versionar el contenido del sitio** en un repositorio Git para disponer de historial de cambios y despliegues reproducibles.

8. **Documentar los procedimientos** (instalación, restauración, altas de usuarios) para que otra persona pueda reproducirlos sin depender del conocimiento tácito.
9. **Realizar pruebas de restauración periódicas** en un entorno separado, no solo en producción.
10. **Mantener el sistema actualizado** con `apt upgrade` programado y revisión de avisos de seguridad.

Anexo A — Estructura del proyecto

Link del repositorio de el proyecto:

```
https://github.com/J0sshuaCh/servidor-web-ubuntu-nginx.git
```

```
TareaTaller/
├── 00-documentacion/
│   ├── informe.md                 Este documento
│   ├── fase1.md                   Detalle de la fase 1
│   ├── bitacora.md                Registro de comandos y resultados
│   └── fase1-instalacion-ubuntu.md Guía de instalación
├── 01-capturas/                   Evidencias gráficas
├── 02-config/                     Configuraciones del servidor
├── 03-backups/                    Copias locales del sitio
├── scripts/                       Scripts de cada fase
│   ├── f1-preflight.ps1
│   ├── f1-crear-vm.ps1
│   ├── f1-arrancar-vm.ps1
│   ├── fase3-nginx.sh
│   ├── fase4-usuarios.sh
│   ├── fase5-monitoreo.sh
│   └── fase6-backup.sh
└── sitio/                         Contenido publicado
    ├── index.html
    ├── estilos.css
    └── favicon.svg
```